

California State Polytechnic University Pomona

Department of Electrical and Computer Engineering

**ECE 4301 Cryptographic Algorithms on
Reconfigurable Hardware**

Midterm 2: TRNG (Rust)

Submitted: 12/3//2025

Class Section: ECE 4301-01

Author: Caleb Pai, Dalton Hoang, Jacob Chow,

Khristian Chan, Nathan Marlow

True Random Number Generators (TRNGs) are used in modern computer systems because they help give unpredictable values which make cryptographic operations a lot more secure. They do this by helping randomize memory layouts and strengthening overall system security. Compared to pseudorandom generators, TRNGs pull randomness from the physical world, which is a lot more random and unpredictable compared to pseudorandom generator algorithms that can technically be reproduced. Even though all TRNGs aim for the same thing (getting reliable entropy), the way they get that entropy is actually very reliant on a number of factors, like the design of the chip, its environment, or even the hardware components chosen to make the chip. Most hardware TRNGs follow a similar structure, however, which is the entropy source being input into a digitizer/sampler, then processed by a signal conditioner, and then eventually the value shows up in the output interface.

There are 5 main types of TRNGs used, and they all use different methods to get entropy. One of the most common types is the Thermal Noise TRNG. This type uses the random movement of electrons within a resistor or diode, which creates this thermal noise. This works, because even when these components are at stable temperatures, the electrons still have an unpredictable movement/jitter, which helps create this TRNG. The thermal TRNG takes the voltage noise (electron jitter), and then amplifies and digitizes it repeatedly, which gives us our random number, and since this type of movement relies on quantum-level effects, it is considered secure to use this method.

Another common TRNG method is the ring oscillator method, which uses the fact that every time an oscillator completes a cycle, there is a little bit of jitter in its timing, and it isn't always exactly the same. This is because a ring oscillator is made up of a bunch of different inverters, which all have their own small timing differences because of things like noise, temperature, or quantum variations. The TRNG can use these random differences in timing to

produce random variables, and can easily produce more by sampling the output of two or more different oscillators at slightly different speeds, and this method is very fast and energy efficient, and also super easy to integrate into a system's digital logic, which is why they are pretty common in crypto accelerators. Because this method is very digital-friendly, it is very popular in FPGAs and ASICs. It scales well, doesn't require sensitive analog components, and lets designers crank up the entropy by adding more oscillators or by adjusting the sampling strategy. However, in an overly stable environment, oscillators can become too synchronized, which lowers entropy, but this can be fixed with good designs that include health tests and whitening functions to clean up any weak or biased outputs. In general, these TRNGs are secure because they are based on random physical factors like random noise and temperature differences, so they are also unpredictable and unable to be easily compromised.

The third type of common TRNG is a metastability TRNG, which capitalizes on the fact that sometimes digital circuits will enter an unstable state because of a race condition, which is when two signals arrive at the exact same time, which puts a circuit into a metastable state, where it can't decide between the two signals. This usually causes a lot of issues, especially in things like flip flops, which need to hold certain values at certain times, and even though most designers try to avoid situations like these, the metastable TRNG actually uses these race conditions to get its random values. Some of these metastable TRNGs try to replicate race conditions to generate randomness, and then use the final value (since it is decided randomly) as its randomly generated value. This is also secure, because the outcome of race conditions are truly random, and are borderline impossible to replicate or predict.

One more common TRNG type that is super interesting is the avalanche/zener noise TRNG. This method uses the breakdown behavior of diode when it is reversed biased in its breakdown region. When this happens, the electric field becomes strong enough to let a bunch of electrons through the barrier, which creates a super noisy current, which is full of spikes/bursts of these electrons. This helps with the security of this method, because these spikes are straight up unpredictable at the physical level, and when amplified and digitized, they provide really good entropy. Avalanche TRNGs also have excellent randomness because the noise source is so strong and can be amplified and digitized very easily. However, the biggest problem they have is that they need specific analog components that aren't always part of a standard digital CMOS

design, which is why they are usually only seen in discrete security modules or older hardware security chips.

The TRNG we implemented on the Raspberry Pi's Crypto Engine follows the same core principles as the TRNG types described above. It is based on ring-oscillator jitter, which is one of the most common digital TRNG methods used in hardware security modules. The Pi's hardware uses small timing differences between oscillators inside the chip to generate entropy, which is basically the same concept behind most digital TRNG designs. When we pull random values through the crypto engine driver, those numbers come straight from this physical timing noise. The hardware then conditions the raw values to remove bias and make the output more stable. When we tested it by printing random samples and doing simple checks, the results matched what you would expect from a jitter-based TRNG, which were unpredictable values that come directly from tiny variations in the hardware. Below are some screenshots of the output when looking at the past 4096 bits output by the TRNG.

```
khristian@khristian:~/crypto_git/ECE4301_Fall12025/Group A/Midterm2/piTRNG $ sudo ./target/release/piTRNG 4096
Reading 4096 bytes from /dev/hwrng ...
First 64 bytes (hex):
7d 83 9d ba 59 be a3 a5 f3 a5 d3 7e 41 a0 56 12
7e 6c 53 fc 79 e6 db db 45 b9 b6 f1 9e 3e 4f 35
b9 ae 6a 32 eb 5a 9c 73 57 52 4b f2 82 7f e9 21
80 26 b6 30 b2 05 52 22 10 4f 68 85 4d bc e5 1a

Monobit stats:
total bits: 32768
zeros: 16203 (49.448%)
ones: 16565 (50.552%)

Byte histogram (value: count for first 16 values):
0: 13
1: 10
2: 11
3: 15
4: 14
5: 15
6: 12
7: 17
8: 12
9: 9
10: 20
11: 15
12: 15
13: 14
14: 13
15: 14
```

```
khristian@khristian:~/crypto git/ECE4301_Fall2025/Group A/Midterm2/piTRNG $ sudo
./target/release/piTRNG 4096
Reading 4096 bytes from /dev/hwrng ...
First 64 bytes (hex):
c1 86 d4 dc a2 6e df c2 52 83 5c 8c f6 37 46 b7
11 ff 58 d1 21 7f 94 53 26 8b 4a c2 59 cf 11 fc
a8 63 e4 f5 5d 38 2d be bf 25 1f b6 14 04 96 45
72 93 4d 15 15 dc 0e 2d d0 04 75 9e f5 4b 39 c9

Monobit stats:
total bits:    32768
zeros:         16369 (49.954%)
ones:          16399 (50.046%)

Byte histogram (value: count for first 16 values):
0: 13
1: 9
2: 18
3: 23
4: 14
5: 15
6: 17
7: 20
8: 15
9: 13
10: 15
11: 19
12: 10
13: 19
14: 18
15: 14
```